



Manual Técnico — Arkana Appointments

Manual Técnico

Autor: Ángel Andrés Villorina Cambero

Ciclo formativo: DAW2-A

Proyecto: Proyecto Final de Grado

Centro educativo: IES Albarregas

Lugar y fecha: Mérida, Badajoz — Junio 2026

Documento de referencia para el tribunal y para futuros desarrolladores. Describe el stack, la base de datos, la estructura del código, los requisitos cubiertos y las acciones disponibles por rol.

1. Stack tecnológico

1.1. Frontend

Tecnología	Versión	Uso
React	19.x	Librería de UI con Suspense y React.lazy para code-splitting por ruta.
TypeScript	5.9.x (modo estricto)	Tipado del proyecto entero.
Vite	7.x	Bundler y servidor de desarrollo.
React Router	7.x	Enrutado en cliente con rutas anidadas y guardas por sesión y rol.
Tailwind CSS	4.x	Estilos utility-first.
Recharts	3.x	Gráficas del dashboard admin y del negocio.
Mapbox GL JS	3.x	Mapa interactivo del perfil del negocio.

Tecnología	Versión	Uso
date-fns	4.x	Manipulación y formateo de fechas en español.
react-qr-code	2.x	Generación del código QR con logo.
react-hot-toast	2.x	Feedback inmediato de acciones.
react-icons (bi)	5.x	Set de iconos Boxicons.

1.2. Backend (BaaS)

Servicio	Uso
Supabase Auth	Registro, login, JWT, verificación de email, recuperación de contraseña.
Supabase Postgres	Base de datos relacional con tipos enum, triggers, índices y RPCs.
Supabase Storage	Buckets <code>avatares</code> , <code>logos</code> , <code>galeria-negocios</code> .
Supabase Realtime	Notificaciones en vivo en el panel del negocio y del cliente.
Edge Functions	<code>send-whatsapp</code> : notificaciones WhatsApp al crear/confirmar/cancelar/reagendar citas.

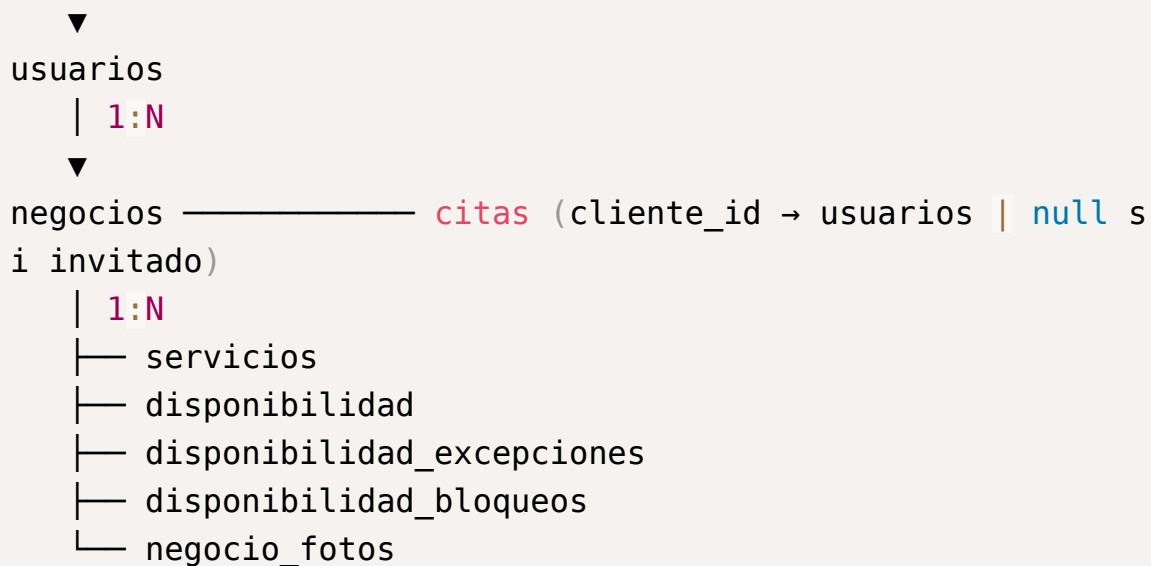
1.3. Infraestructura

Componente	Proveedor
Hosting frontend	Vercel (Hobby, CI/CD desde <code>main</code> , previews por rama)
Backend y BD	Supabase (Free, región EU)
Mapas	Mapbox (token público restringido por dominio)
WhatsApp	Evolution API en Railway (~5 USD/mes)
Dominio	<code>www.arkana-appointments.com</code>

2. Modelo de datos

2.1. Diagrama lógico

```
auth.users (Supabase)
| 1:1
```



2.2. Tablas principales

Tabla	Propósito
<code>usuarios</code>	Perfil aplicativo (rol, email, teléfono, foto, <code>eliminado</code>).
<code>negocios</code>	Negocio publicado (slug, lat/lng, teléfono, <code>whatsapp_activado</code>).
<code>servicios</code>	Catálogo de servicios por negocio.
<code>disponibilidad</code>	Horario semanal por día.
<code>disponibilidad_excepciones</code>	Aperturas puntuales fuera del horario regular.
<code>disponibilidad_bloqueos</code>	Días específicos cerrados.
<code>negocio_fotos</code>	Galería del negocio.
<code>citas</code>	Reserva (negocio, servicio, cliente, fecha, hora, estado).
<code>notificaciones</code>	Notificaciones persistentes por usuario.
<code>whatsapp_log</code>	Registro de cada mensaje WhatsApp enviado.

2.3. Tipos enum

```

CREATE TYPE rol_usuario AS ENUM ('cliente', 'negocio', 'admin');
CREATE TYPE estado_cita AS ENUM ('new', 'pending', 'confirmed', 'cancelled', 'completed');

```

2.4. RPCs (SECURITY DEFINER)

Función	Descripción
<code>admin_create_usuario(...)</code>	Crea usuario en <code>auth.users</code> • <code>auth.identities</code> • <code>public.usuarios</code> (con email ya verificado). Opcionalmente crea negocio si el rol es <code>negocio</code> .
<code>admin_delete_usuario(p_user_id)</code>	Borra al usuario completamente: negocios (cascade), <code>public.usuarios</code> y <code>auth.users</code> en una sola transacción.
<code>es_admin()</code>	Helper interno; devuelve <code>true</code> si el llamante tiene rol <code>admin</code> .

2.5. Índices de rendimiento

- `idx_citas_negocio_fecha` — `(negocio_id, fecha)`
- `idx_citas_negocio_estado` — `(negocio_id, estado)`
- `idx_disponibilidad_negocio_dia` — `(negocio_id, dia_semana)`
- `uq_citas_slot` — índice único parcial `(negocio_id, fecha, hora_inicio) WHERE estado <> 'cancelled'`

3. Estructura del código (`src/`)

```
src/
├─ App.tsx                # Routing, lazy loading y error boundaries globales
├─ main.tsx               # Entry point con AuthProvider y NotificationsProvider
├─ assets/                # Logos SVG, fuentes SF Pro
├─ components/
│   └─ app/               # Componentes compartidos (Sidebar, Spinner, Modales, Mapas...)
│       └─ blocks/        # Bloques de la landing (Navbar, Hero, Footer)
│           └─ ui/         # Primitivos UI (Sheet, Accordion, Button, ThemeToggle...)
├─ contexts/
│   └─ AuthContext.tsx    # Sesión, perfil, negocio y métodos de auth
│       └─ NotificationsContext.tsx # Realtime + polling de re
```

```

spaldo cada 10 s
├─ hooks/                                # Hooks personalizados (use
Theme, etc.)
├─ layouts/
│   ├─ AppLayout.tsx                    # Panel del negocio (guard
a de rol + email verificado)
│   ├─ ClienteLayout.tsx                # Portal del cliente (guar
da + email verificado)
│   └─ AdminLayout.tsx                  # Panel admin
├─ lib/
│   ├─ supabase.ts                      # Cliente Supabase + tipos
del dominio
│   └─ validators.ts                    # Validaciones (email, con
traseña, teléfono)
├─ utils.ts                             # Helpers generales
└─ pages/
    ├─ arkana/                          # Landing pública
    ├─ auth/                            # Login, registro, recuper
ar/nueva contraseña, verificar email
    ├─ booking/                         # Página pública /n/:slug
    ├─ cliente/                         # Portal del cliente (/ap
p)
    ├─ dashboard/                       # Panel del negocio (/pane
l)
    ├─ admin/                           # Panel admin (/admin)
    └─ legal/                           # Privacidad, Términos, Es
tado, Seguridad, Sobre nosotros

```

4. Flujos de autenticación

Flujo	Descripción
Registro	Email + contraseña. Supabase envía email de verificación. La app bloquea el acceso hasta que se confirme.
Login	Email + contraseña vía Supabase Auth. Redirige según rol (<code>/panel</code> , <code>/app</code> , <code>/admin</code>).
Recuperación	El usuario solicita enlace en <code>/recuperar-password</code> . Supabase envía email con token. El enlace lleva a <code>/nueva-password</code> donde

Flujo	Descripción
	introduce la nueva contraseña.
Cambio de email	Desde Configuración. Supabase envía confirmación al nuevo email antes de aplicar el cambio.
Cambio de contraseña	Desde Configuración. Supabase envía un enlace seguro al correo actual para elegir nueva contraseña.
Admin crea usuario	Usa RPC <code>admin_create_usuario</code> . El usuario queda con email ya verificado y puede iniciar sesión de inmediato.

5. Acciones disponibles por rol

5.1. Cliente

- Reservar en cualquier negocio publicado (con o sin cuenta).
- Ver y cancelar sus citas próximas.
- Recibir notificaciones en tiempo real.
- Editar perfil (nombre, teléfono, foto).
- Cambiar email y contraseña desde Configuración.
- Cambiar tema claro / oscuro.

5.2. Negocio

- Editar perfil público completo (logo, galería, ubicación en mapa, servicios).
- Definir horario semanal, excepciones y bloqueos de días.
- Ver y gestionar citas en tiempo real (confirmar, completar, cancelar, reagendar).
- Crear una cita manualmente.
- Descargar código QR.
- Consultar estadísticas (hoy / semana / mes + tendencia 12 días).

5.3. Administrador

- Dashboard con métricas agregadas y gráfica de altas a 30 días.
- Listar, buscar, filtrar, crear, editar, desactivar/activar y eliminar usuarios.
- Crear negocio completo (usuario + perfil) desde la UI.

Cómo desplegar el proyecto desde cero

1. Requisitos

- Node.js $\geq$ 20.10
- npm $\geq$ 10
- Cuenta gratuita en [Vercel](#)
- Cuenta gratuita en [Supabase](#) (región EU)
- Cuenta gratuita en [Mapbox](#) para el token de mapas

2. Clonar y configurar

```
git clone https://github.com/glitterboy16/Arkana-Appointment.git
cd Arkana-Appointment
npm install
cp .env.example .env.local
```

Rellenar `.env.local` con tres valores:

```
VITE_SUPABASE_URL="https://<tu-proyecto>.supabase.co"
VITE_SUPABASE_ANON_KEY="<anon-key-publica>"
VITE_MAPBOX_TOKEN="pk.<tu-token-de-mapbox>"
```

3. Base de datos

En el proyecto de Supabase, abrir el **SQL Editor** y ejecutar **en orden** las migraciones de la carpeta `supabase/migrations/`. El número de archivo indica el orden:

Archivo	Contenido
<code>001_esquema_completo.sql</code>	Columnas, tablas auxiliares, RLS, Storage y Realtime
<code>002_admin_panel.sql</code>	Rol admin, RPCs y usuario administrador inicial
<code>003_whatsapp_notificaciones.sql</code>	Opt-in WhatsApp, log de envíos y notificaciones persistentes

Archivo	Contenido
<code>004_unique_slot.sql</code>	Índice único que impide solapamiento de citas activas

4. Despliegue en Vercel

1. Importar el repo desde Vercel.
2. Framework: **Vite** (autodetectado).
3. Build command: `npm run build`. Output: `dist`.
4. Variables de entorno → marcar los 3 environments (Production, Preview, Development) para cada una.
5. Asignar dominio: `www.arkana-appointments.com`.

A partir de aquí, cualquier push a `main` despliega producción y cualquier push a una rama `feature/*` genera un preview deployment.

5. Verificación

- Abrir la URL pública → debe cargar la landing.
- Iniciar sesión con un usuario de prueba → debe redirigir al panel correspondiente.
- Crear una cita desde un negocio publicado → debe aparecer en el panel del negocio en tiempo real.

##
